Project Smart Trigger Design Documentation for ECE/CS 3992

Andrew Koebbe *Student ID: u1450645*, Stephen Hofmann *Student ID: u1028605*,
Thomas Stay *Student ID: u1249316*, Derek Jenkins *Student ID: u1506428*

***Abstract*—This document describes the design of the Smart Trigger, a modular RFID-based access-control system for the AR-15 platform. A passive 125kHz RFID tag worn by the authorized user is read by an antenna embedded in a custom 3D-printed grip. A Raspberry Pi Pico microcontroller validates the tag against an authorized list and drives a micro stepper motor that moves a blocking member out of the fire control group. The document covers the system architecture, the hardware components and their integration onto a custom PCB, the CAD modeling of the grip and actuator mounts, and the firmware processes that tie the system together. It closes with a look at how the design could grow through an off-device logging server and web application, and what that complexity would cost in power, security, and development effort.**

I. Introduction

II. System Architecture

A. Overview

The Smart Trigger is a self-contained, battery-powered embedded system that adds an electronic authorization step to the mechanical operation of an AR-15. Functionally, it is a closed loop with three stages: Sense, Decide, Act.

B. Subsystems

C. Design Requirements

D. Key Architectural Decisions

III. Hardware Components and Integration

A. Mechanical Interaction

The mechanical design has three parts:

1) a **custom grip** that replaces the factory AR-15 grip and houses the electronics, battery, and RFID antenna
2) an **actuator mount** that locates the micro stepper in the fire control pocket of the lower receiver
3) a **blocking member** driven by the motor that physically prevents the fire control group from releasing the hammer

TABLE I
Components in the Smart Trigger system

Part Number	Short Name	Short Description
Pi Pico W	Pico	Microcontroller and central brain
SAA1042	Motor Driver	Stepper motor driver
RDM6300	RFID Reader	RFID reader and translator
ANT1	Antenna	Custom wound antenna tuned to 125kHz
M1	Motor	Stepper motor to rotate shaft
R1	R	47kΩ resistor
C1	C	100 nF capacitor

To understand the innovation behind the blocking member takes some understanding of the original function of the AR-15 fire control group(trigger, hammer, sear, and safety). To fire a round the trigger must be pulled, the angle the trigger is brought to in its movement releases the hold it had on the spring loaded hammer. The freed hammer slams against onto a hook called the disconnector which holds the hammer until the trigger is back to its ready position it was in before firing the last round. The safety on the AR-15 is a physical barrier to the trigger. When is the safe position the trigger is physically blocked from movement thus preventing the hammer from moving and the overall firing of the gun. The design of our safety takes advantage of the mechanical blocker of the AR-15's native safety selector. To create the most widely applicable design we focused on locking the native safety in place until an authorized user is recognized.

B. Component Selection

C. Main Board (PCB)

1) Rationale for a Custom PCB: The project imposed a strict size constraint on the electronics enclosure: the pistol grip measures [insert grip dimensions]. A custom printed circuit board (PCB) would integrate all components listed in Table **??** into a compact form. A custom design also offered greater control over the system, since unnecessary components could be removed and only the

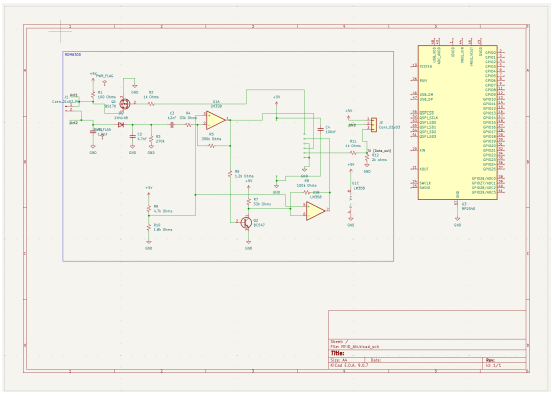


Fig. 1. First Iteration of the Custom PCB Design. The design integrates the RDM6300 RFID module, the SAA1042 stepper motor driver, and the Raspberry Pi Pico microcontroller into a single board.

required circuitry selected from each of the commercial development boards. Finally, a single integrated board would simplify installation for the end user compared with a multi-board assembly.

2) Design Process and Change in Approach: With these constraints in mind, we began by reverse engineering the RDM6300 RFID module, which serves as the interface between the Raspberry Pi Pico microcontroller and the RFID tag. We recreated the circuit in KiCad, a PCB design tool, and started to validate it through simulation in LTSpice. The resulting design is shown in Figure **??**.

This design would have allowed us to remove the RP2040 from its development board and mount it directly on the custom PCB, eliminating the many GPIO pins we did not intend to use.

Before fabricating and testing the board, however, the team questioned whether the fine-pitch leads of the RP2040 could be reliably hand soldered. Commercial manufacturers typically place such components using processes such as stencil-based solder paste application with reflow, or thermocompression bonding, neither of which was available to us. Given the project schedule and the risk of failing to mount the chip, we elected to pursue a simpler design.

3) Tradeoffs of the Revised Design: Although each of the existing boards fit within the grip, they would not be held securely in place. We therefore changed the role of the PCB to that of a perfboard-style central hub that interconnects the individual boards. This decision involved several tradeoffs. The perfboard approach retains unused features, including GPIO pins, clock circuitry, and UART pins. The boards must also be arranged to fit within the grip, making the assembly less compact than a custom PCB that integrates their functions. However, this approach required no additional parts, since all components were already in hand, and its lower complexity improved the likelihood of delivering a working product by the deadline.

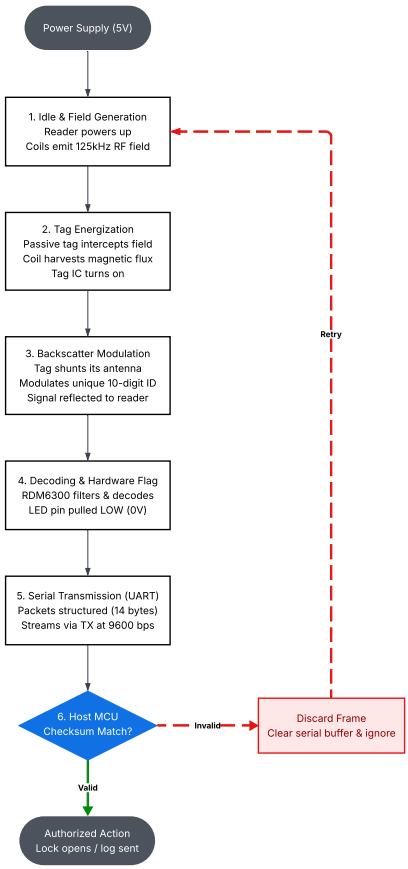


Fig. 2. A flow diagram of the RDM6300 RFID module. The module reads the RFID tag and translates its signal into a UART serial output for the Raspberry Pi Pico.

The current design is shown in Figure . The board is configured to sandwich the Raspberry Pi Pico microcontroller on one side with the motor driver on the opposite side. The RDM6300 sits directly below the motor driver and is mounted through solder points that accept the short portion of its header pins, forming a compact stack of boards.

D. RFID Board (RDM6300)

E. Micro Stepper Motor & Driver Board (SAA1042)

F. Integration and Test Plan

IV. CAD Modeling and Design

A. Scope of the Mechanical Design

B. Custom Grip

C. Actuator Mount and Blocking Member

D. Materials and Print Parameters

E. Design Iteration

V. Software Processes

A. Development Environment

The development environment for the firmware is based on the Raspberry Pi Pico SDK (version 2.0 or greater), which provides a comprehensive set of tools and libraries for programming the RP2040 microcontroller on the Raspberry Pi Pico W. The firmware is written in C11 and utilizes the FreeRTOS SMP kernel with Heap 4 memory management to manage tasks and ensure responsive, deterministic performance. Build configuration and cross-compilation are handled via CMake, targeting both USB serial and an independent hardware UART debug interface.

B. Firmware Architecture

The firmware architecture is designed to be modular and maintainable, with each component having a specific role and responsibility divided across isolated compilation units (‘main.c’, ‘rfid.c’, and ‘stepper.c’). The system is divided into several layers, including the hardware peripheral access layer (Pico SDK hardware drivers), an RTOS task synchronization layer, and the application logic layer. This design allows for easier debugging and maintenance while ensuring that the system remains robust under typical usage scenarios by preventing driver operations from blocking critical timing loops.

C. Task Management and Inter-Task Synchronization

Rather than relying on a polling super-loop, the firmware implements four concurrent FreeRTOS tasks to manage system states and hardware operations: ‘rfid_task’, ‘stepper_task’, ‘led_task’, and ‘console_task’. Inter-task communication and hardware decoupling are handled through kernel synchronization primitives:

- UART Interrupt and Stream Buffer: A dedicated UART interrupt service routine (‘rfid_uart_irq’) drains incoming reader bytes directly from the 32-byte hardware FIFO into an RTOS stream buffer (‘rx_stream’), eliminating byte drops caused by serial print latency in application tasks.
- Task Notifications: When ‘rfid_task’ validates an authorized tag, it signals ‘stepper_task’ using lightweight FreeRTOS direct task notifications (‘xTaskNotifyGive’ and ‘ulTaskNotifyTake’) via ‘stepper_start()’, and halts rotation with ‘stepper_stop()’ when the tag is removed.
- Decoupled Actuator and LED Scheduling: ‘stepper_task’ signals ‘led_task’ on each completed step pulse via task notifications, decoupling the timing of wireless chip I/O from physical motor control.

D. RFID Processing

The RFID processing component is responsible for reading and interpreting data from an external RDM6300 125kHz RFID module operating at 9600 baud (8N1) over UART0 on GP1. It handles communication with the reader, processes incoming serial bursts, and determines appropriate actions based on tag information:

- Frame Demarcation: The parser buffers data bounded by an STX (0x02) start byte and an ETX (0x03) end byte, capturing a 10-character ASCII hex payload and a 2-character checksum.
- Checksum Verification: The module verifies the RDM6300 XOR checksum across the five decoded data bytes before any access evaluation occurs.
- Authentication and Presence Timeout: The decoded tag is matched against a pre-compiled identifier macro (‘RFID_VALID_TAG’, defaulting to EM410x tag ‘BEAD101000’). If valid, the system maintains trigger activation while tag frames continue to arrive, halting rotation if no valid frame is received within a 500 ms timeout window (‘RFID_GONE_MS’) or if an unauthorized tag is detected.

E. Actuator Control

The actuator control component manages the physical operation of an Abovehill 8 mm micro stepper motor driven by an SAA1042 stepper controller IC in half-step mode (20 full steps/rev). It receives commands from ‘rfid_task’ and translates them into specific actions:

- Pulse Generation: ‘stepper_task’ outputs a 10 ms active-high step pulse (‘STEP_PULSE_WIDTH_MS’) on GP22.
- Deterministic Periodicity: Pulse cadence is locked to a 100 ms interval (‘STEP_PERIOD_MS’) using ‘vTaskDelayUntil’, providing consistent mechanical stepping without timing drift.

- Directional Control: Direction is set to clockwise (‘STEPPER_TRIGGER_CW’) by asserting GP21 prior to stepping.

F. Hardware Interface and Pad Configuration

The firmware directly addresses the physical electrical constraints of the peripheral components:

- 5V to 3.3V Logic Interfacing: Because the RDM6300 module transmits at 5V and RP2040 GPIO pads tolerate a maximum of 3.3V, serial data is received through a 2kΩ series resistor on GP1, with dynamic software pull-downs configured for connection diagnostics.
- Slew Rate and Pad Drive Strength: To ensure clean switching thresholds across the SAA1042 input logic, GP21 (direction) and GP22 (clock) are programmed for fast slew rates and an increased 12 mA drive strength.
- CYW43 Bus Isolation: On the Raspberry Pi Pico W, the onboard status LED is driven via the Infineon CYW43439 wireless chip over a shared internal bus. Because communicating with the CYW43 involves SPI transactions that introduce timing latency, LED state changes are isolated to a dedicated low-priority background task (‘led_task’), preventing motor step jitter.

G. Software Testing and Hardware Diagnostics

Testing and diagnostic instrumentation are integrated directly into the firmware architecture:

- Integrated Self-Test Modes: Using the ‘RFID_SELFTEST’ build flag, the firmware can run full automated regression tests without physical tags. Mode 1 activates the internal hardware UART loopback inside the RP2040 (‘UART_UARTCR_LBE_BITS’), while Mode 2 configures GP0 as an external transmitter jumpered to GP1 to test pad and trace continuity. A synthetic test task (‘rfid_test_task’) transmits alternating valid and invalid framed packets every 2000 ms to verify checksum logic and driver triggering.
- Line State and Bus Diagnostics: A periodic diagnostics routine polls the physical line state of GP1 and reports hardware-level UART errors, tracking framing errors, line breaks, and FIFO overruns on the debug console.
- Telemetry Console: Diagnostic logging is broadcast over a dedicated secondary UART (UART1 on GP4 at 115200 baud) and concurrent USB CDC serial (‘stdio_usb’), isolating debug telemetry from the RFID module interface.

H. Future Work: Networking and Telemetry Integration

The Smart Trigger system architecture is prepared for expansion into network telemetry and centralized access auditing using the onboard CYW43439 wireless chip of the Raspberry Pi Pico W. While the present implementation links ‘pico_cyw43_arch_sys_freertos’ strictly for onboard LED control with the lwIP stack disabled (‘CYW43_LWIP=0’), the underlying FreeRTOS environment is already structured to support networking. Transitioning to an active network node requires:

1) Enabling the lwIP TCP/IP stack (‘CYW43_LWIP=1’) within the CMake build target to enable socket-level networking.
2) Implementing a secure FreeRTOS client task (using MQTT over TLS or HTTPS) to publish access logs, timestamped valid reads, and tamper/break alerts to an offloaded database server.
3) Deploying a web service dashboard to authenticate administrators and permit remote over-the-air updates to the authorized tag whitelist (‘RFID_VALID_TAG’).

VI. Conclusion

References